Universidad Icesi

Autenticación y archivos con Supabase

Quién eres, qué puedes hacer y dónde viven las fotos

Seminario de I+D+I — Universidad Icesi

Mapa de la sesión



Autenticación

Demostrar quién eres: registrarse e iniciar sesión



Autorización

Decidir qué puede hacer cada usuario con RLS



Archivos

Guardar fotos en Storage y mostrarlas en la app

Universidad ICESI

Autenticación

Quién eres y qué puedes hacer

Dos preguntas distintas

Autenticación: ¿quién eres?

Comprobar la identidad con correo y contraseña. La resuelve **Supabase Auth** y entrega una sesión.

Autorización: ¿qué puedes hacer?

Decidir qué filas y archivos ve cada identidad. La resuelven las **políticas RLS** de la base de datos.

Dos puertas en fila



Quien llega

la app con un usuario

Alguien llega a la app. Todavía no sabemos quién es.

Dos puertas en fila



Quien llega
la app con un usuario

Autenticación
¿quién eres?

Primero la autenticación: demuestra su identidad y recibe un carné.

Dos puertas en fila



Después la autorización: el carné decide qué filas puede ver.

El carné: un JWT



Al iniciar sesión, la app envía correo y contraseña a Supabase Auth.

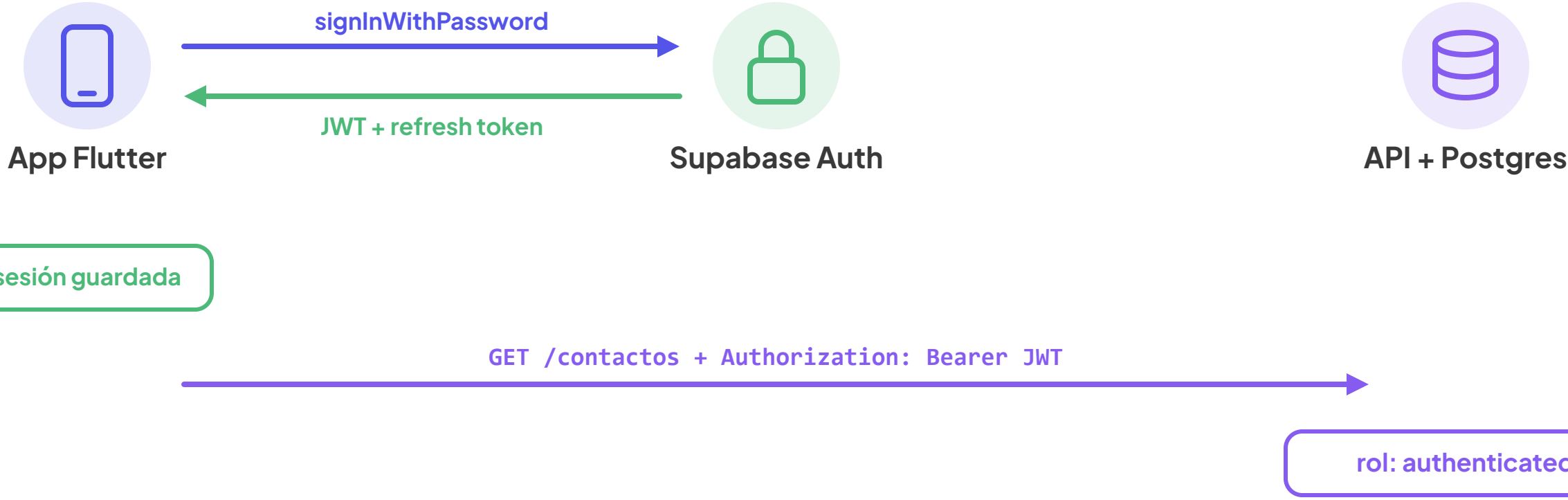
El carné: un JWT



sesión guardada

Auth responde con un JWT. El SDK lo guarda y lo renueva solo.

El carné: un JWT



Desde entonces, cada petición lleva el JWT: Postgres sabe quién pide.

Los métodos del SDK

Método	Qué hace	Si falla
`signUp(email, password)`	Crea el usuario	AuthException
`signInWithPassword(...)`	Comprueba la contraseña y abre la sesión	AuthException
`signOut()`	Cierra la sesión y borra el JWT guardado	—
`currentSession`	La sesión actual, sin ir a la red	null si no hay
`onAuthStateChange`	Un Stream: avisa al iniciar o cerrar sesión	—

Un servicio para la autenticación

lib/services/auth_service.dart

```dart
class AuthService {
  final _auth = Supabase.instance.client.auth;

  Future<void> iniciarSesion(String email, String pass) async {
    await _auth.signInWithPassword(email: email, password: pass);
  }

  Future<void> cerrarSesion() => _auth.signOut();
}
```

Un solo cliente

client.auth es el módulo de autenticación del mismo cliente de la semana 14.

Un servicio para la autenticación

lib/services/auth_service.dart

```dart
class AuthService {
  final _auth = Supabase.instance.client.auth;

  Future<void> iniciarSesion(String email, String pass) async {
    await _auth.signInWithPassword(email: email, password: pass);
  }

  Future<void> cerrarSesion() => _auth.signOut();
}
```

Un método por operación

Las pantallas llaman al servicio y no importan **supabase_flutter**.

La guarda decide la pantalla

lib/auth_gate.dart

```dart
StreamBuilder<AuthState>(
  stream: servicio.cambios,
  builder: (context, snapshot) {
    final sesion = snapshot.data?.session ?? servicio.sesion;
    return sesion == null ? const LoginPage() : const ContactosPage();
  },
)
```

Escucha la sesión

onAuthStateChange emite cada vez que alguien entra o sale.

Universidad ICESI

La guarda decide la pantalla

lib/auth_gate.dart

```dart
StreamBuilder<AuthState>(
  stream: servicio.cambios,
  builder: (context, snapshot) {
    final sesion = snapshot.data?.session ?? servicio.sesion;
    return sesion == null ? const LoginPage() : const ContactosPage();
  },
)
```

`Sin Navigator`

Con sesión: la agenda. Sin sesión: el ingreso. La pantalla cambia sola.

Universidad ICESI

De anon a authenticated

Qué rol trae cada petición

Rol de la petición y política



Funciona, pero cualquiera con la llave publishable entra.

Rol de la petición y política



Al iniciar sesión dejas de ser anon: la política ya no aplica.

Rol de la petición y política



Solo entra quien tiene sesión.

La política nueva

```sql
create policy "autenticados: leer"
  on public.contactos
  for select
  to authenticated
  using (true);
```

Cambia solo el rol

«to anon» pasa a «to authenticated».
Sin sesión ya no se lee ni se escribe nada.
Con **auth.uid()** cada usuario puede ver solo sus filas.

Universidad ICESI

Archivos con Storage

Subir una foto y mostrarla

Buckets, objetos y reglas

Bucket

Una carpeta de primer nivel con nombre y reglas propias, como «fotos».

Objeto

Un archivo dentro del bucket: imagen, video, audio, PDF. Su nombre puede incluir carpetas.

Reglas

Público o privado, tamaño máximo y tipos permitidos (image/*).

Público abre solo la lectura

Operación	¿Quién puede?	Por qué
Ver una foto con su URL	Cualquiera	El bucket es público
Subir una foto	Usuarios con sesión	Política RLS de insert
Reemplazar o borrar	Nadie, todavía	Sin política de update ni delete

El archivo va a Storage, la ruta a la tabla

App Flutter

Storage
bucket «fotos»

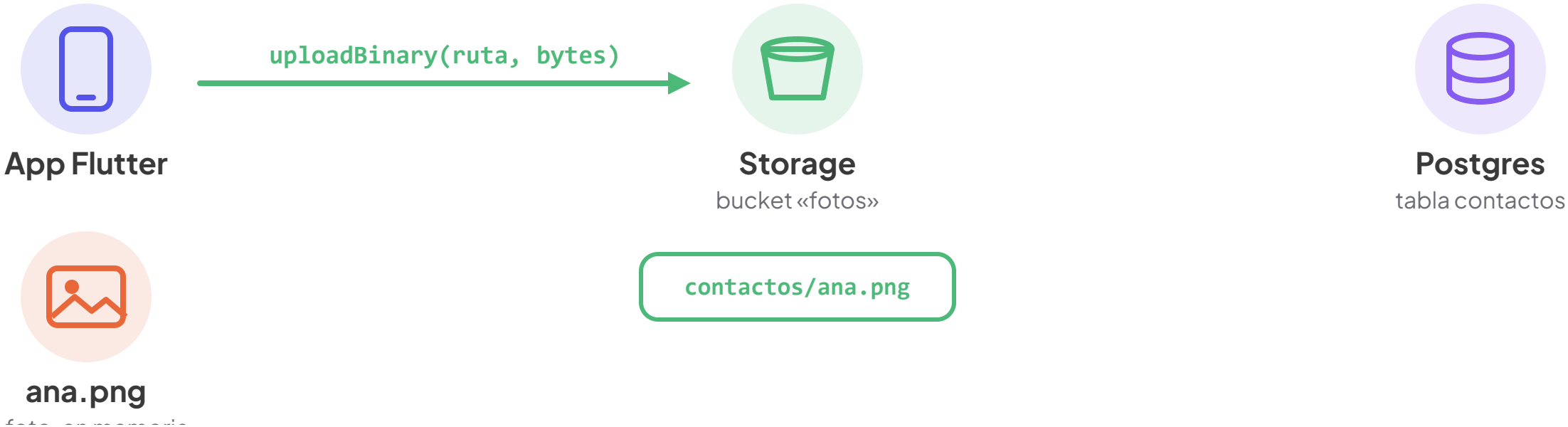
Postgres
tabla contactos



ana.png
la foto, en memoria

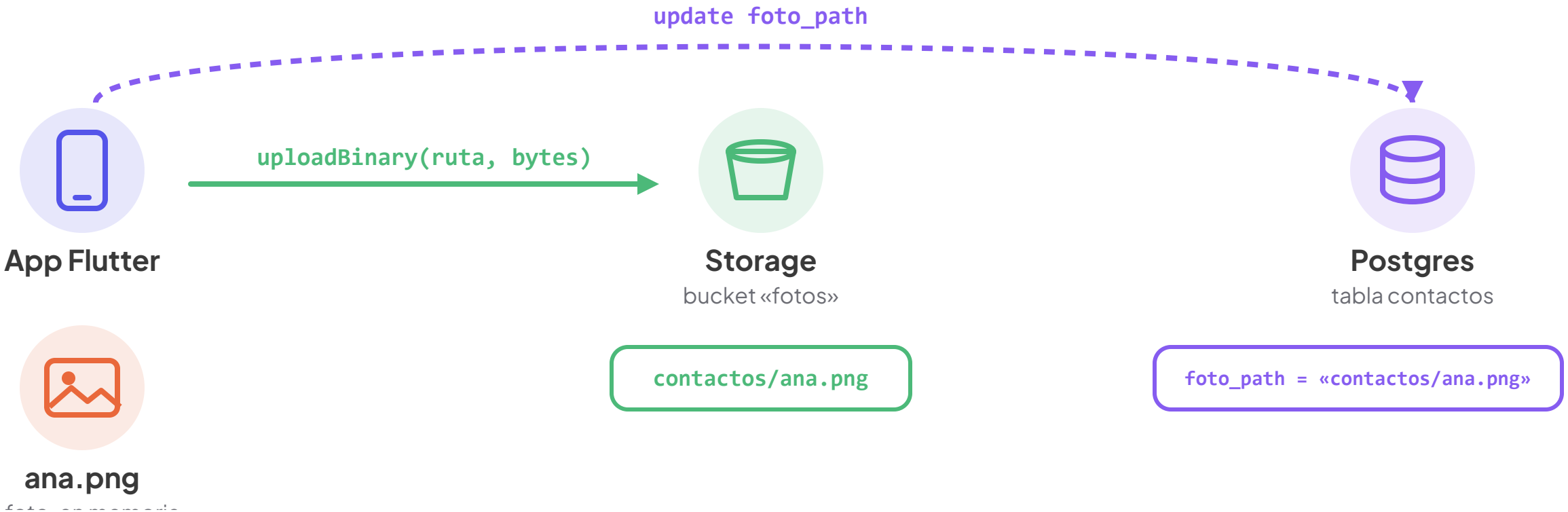
La foto elegida solo existe en la memoria de la app.

El archivo va a Storage, la ruta a la tabla



Se sube a Storage. Eso devuelve una ruta dentro del bucket.

El archivo va a Storage, la ruta a la tabla



La tabla guarda la ruta, no la imagen: una columna de texto.

Subir con uploadBinary

lib/services/foto_service.dart

```dart
final bucket = Supabase.instance.client.storage.from('fotos');

Future<String> subir(XFile foto) async {
  final bytes = await foto.readAsBytes();
  final ruta = 'contactos/${DateTime.now().millisecondsSinceEpoch}_${foto.name}';
  await bucket.uploadBinary(ruta, bytes);
  return ruta; // esto se guarda en foto_path
}
```

Bytes y bucket

from('fotos') elige el bucket, como from('contactos') elige la tabla.
readAsBytes lee la imagen elegida.

Subir con uploadBinary

lib/services/foto_service.dart

```dart
final bucket = Supabase.instance.client.storage.from('fotos');

Future<String> subir(XFile foto) async {
  final bytes = await foto.readAsBytes();
  final ruta = 'contactos/${DateTime.now().millisecondsSinceEpoch}_${foto.name}';
  await bucket.uploadBinary(ruta, bytes);
  return ruta; // esto se guarda en foto_path
}
```

Nombre único

La hora evita pisar fotos con el mismo nombre. **uploadBinary** funciona también en Flutter Web.

Mostrar la imagen

Obtener la URL

```
String urlPublica(String ruta) =>
    bucket.getPublicUrl(ruta); // sin red

Image.network(
  FotoService().urlPublica(c.fotoPath!),
  errorBuilder: (_, __, ___) => const Icon(Icons.broken_image),
)
```

Solo arma texto

getPublicUrl concatena proyecto, bucket y ruta. No usa la red.

Mostrar la imagen

Dibujar la foto

```
String urlPublica(String ruta) =>
    bucket.getPublicUrl(ruta); // sin red

Image.network(
  FotoService().urlPublica(c.fotoPath!),
  errorBuilder: (_, __, ___) => const Icon(Icons.broken_image),
)
```

Sin llave ni sesión

Image.network hace un GET normal a la URL pública. **errorBuilder** cubre rutas malas o falta de red.

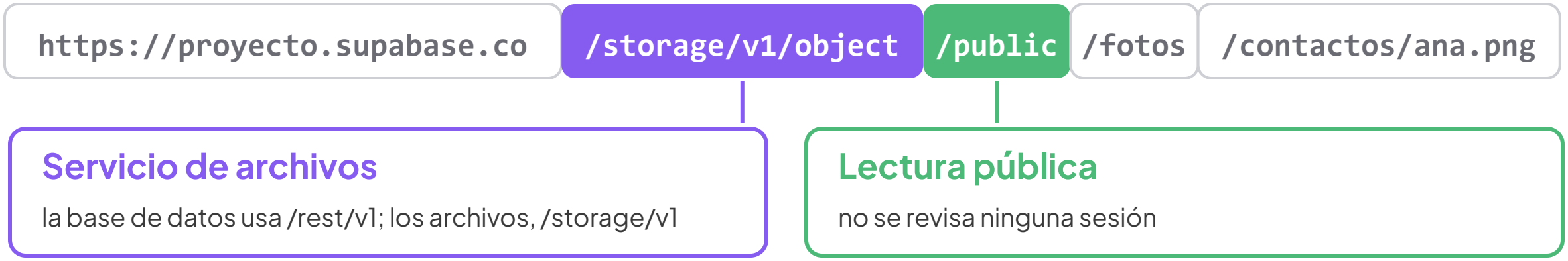
Cómo se lee la URL de un archivo



Tu proyecto

la misma URL base de Supabase.initialize

Cómo se lee la URL de un archivo



Cómo se lee la URL de un archivo



Errores de Storage

Código	Significa	Causa típica
`403`	Rechazado por RLS	Falta la política de subida o no hay sesión
`415`	Tipo no permitido	Subir un PDF a un bucket de image/*
`413`	Archivo demasiado grande	La foto pesa más que el límite del bucket
`409`	El recurso ya existe	Se subió dos veces la misma ruta

Lo que te llevas

Autenticar

Supabase Auth responde «¿quién eres?» y entrega un JWT.

Autorizar

RLS decide por rol: anon no es authenticated.

Archivos aparte

El archivo va a Storage; la tabla guarda su ruta.

URL pública

Se arma con getPublicUrl y se muestra con Image.network.

Universidad Icesi

Preguntas

Semana 15

Seminario de I+D+I — Universidad Icesi